

CLAO

v0.2 全面架构审计 与 v0.3 产品规划

从“闭环已经跑通”走向“结果可信、异常可解释、普通用户可用”。

总评：v0.2 已完成本地比赛版的主路径与净化交付；v0.3 应做用户产品化，而不是重写控制架构。涉及权限、路径边界和终局误判的缺口，建议先作为 v0.2.1 小补丁处理。

审计项	本次基线
产品	CLAO / Closed-Loop Agent Orchestrator, Windows 本地比赛版
发布版本	v0.2; 发布日 2026-09-05
源码提交	4d3e8e6b5e70bab868b2eef0d28c7742dea044ba
证据范围	发布元数据、冻结源码静态审查、历史验收记录、原始报告、隔离函数探针
报告日期	2026-09-05; 本报告不是新版本实现或验收证明

决策摘要

现在保留什么：MissionController + StateStore + AO Worker，默认单 Worker、gate-first、Mission 终局 Verifier、独立发布包。

下一版优先做什么：权限/终局判定补强 → 配置与诊断一致 → 任务导向 GUI 与结果交付 → 经测试的第二种模型供应商。

不应继续宣称什么：所有历史问题已清零、任意仓库都安全、所有“OpenAI compatible”模型都能直接替换、GUI PASS 等于安全测试通过。

来源标记 [Sxx] 见文末。所有 v0.3 设计和验收指标均为规划建议；发布版事实与规划已分栏说明。

01 阅读导航与审计方法

章节	重点
02—05	发布基线、真实架构、已实现功能、原报告闭环
06—11	新增正确性/安全发现、状态恢复、配置、GUI 与性能审计
12—14	多模型切换现状、接入方案、v0.3 用户体验与界面规划
15—18	版本排期、任务清单、验收门禁、指标与发布治理
附录 A—C	隔离复演结果、文档解释、可追溯来源索引

本轮做了什么

固定审查已发布 v0.2，而不是队员的另一分支或未来 main。检查入口组装、Controller/ClosedLoop、AO 读写、审批、Git/Gate、结果契约、事件/存储、Panel 后端与前端、配置、发布说明；并对关键分支做无副作用隔离复演。[S01][S04—S24][S28]

证据等级

标记	含义与边界
源码确认	可在冻结源码中定位到具体类/函数/条件；不自动等于所有运行环境已实测。
隔离复演	提取原函数、SQL 或判定条件，在临时环境复演。未运行 AO、真实模型或浏览器攻击。
已有验收	开发方在本机进行的 438 项回归、CLI live 与用户手动 GUI；本轮核对记录，未重新执行。
推断 / 待验证	由调用顺序或边界推导出的风险，尚需产品级/Windows 场景确认。
规划建议	v0.2.1 / v0.3 目标、设计及验收标准，不属于现有功能。

本轮没有访问用户本机 runtime、认证 profile 或 AO 数据库，没有重新下载验证发布 ZIP 的所有字节，没有进行全项目 Windows 回归、渗透测试或跨供应商 live benchmark。发布 API 摘要、用户人工确认和源码分析各自承担不同证明责任。

审计尺度：一次成功的端到端任务证明这条样例路径可达；它不能证明所有失败分支、权限输入、并发窗口和模型输出都安全。发现边界缺口并不否定已完成的发布验收，但会改变扩大使用范围之前的优先级。

02 v0.2 发布与验收事实

项目	已有的证据
发布	GitHub Release v0.2 已公开；不是 draft；两个用户附件。
发布 ZIP	clao-v0.2-4d3e8e6b5e70.zip；245,995 bytes。
SHA-256	73397cb066f6991681dc8404b1a85c10e80b1169a8a85b278b88ee7bdf035986
包内容	92 个 manifest 产品文件 + SHA256SUMS.txt；顶层 clao/。
已验证环境	Windows；CPython 3.12.7；Git 2.55.0；AO Desktop 0.12.9；Codex CLI 0.150.1；ChatGPT 登录；gpt-5.6-sol。
离线与安装	干净解压包 bootstrap、438 项测试、compileall 和 checksum 均由开发方验证通过。
CLI live	MISSION-R5-FINAL-CLI-20260905-101005，MISSION_DONE，exit 0，418.4 秒。
GUI live	MISSION-PANEL-20260905-103636，用户手动确认 + 后台证据一致，MISSION_DONE。

两次最终任务均采用单 Worker；decomposition Planner、Completion Auditor、completion Planner、Task Verifier 均为 0，Mission Verifier PASS 记录为 1。Task Gate 与 Final Gate 通过；目标 main、origin/main、bare origin 没有自动写回。以上来自已提交验收记录与用户反馈，不是本轮重新实测。
[S01][S03]

交付质量已有实质改善

产品不再包含 AO 安装文件、开发 profile、历史 sidecar、.venv 或 runtime；使用 clean HEAD + SOURCE → DESTINATION allowlist 构建，开发仓库层级不再泄漏到产品目录。bootstrap 只重建 Python 环境，AO/Codex/Git 仍是外部前提。[S23][S24]

定位应保持：已发布、可在支持环境运行的 Windows 本地闭环产品。不能等同于跨平台安装器、云服务、任意项目无人值守平台，亦不能将模型反作弊复核称为安全认证。

03 当前真实架构：保留主干

v0.2 已经形成可维护的主干：Panel 与 CLI 共用 runtime 组装；MissionController 负责 Mission，ClosedLoop 负责子任务；SQLite 保存控制状态；AO 提供 Worker/session/workspace；Bus 和 Markdown 仅投影。[S04—S06][S13][S16][S19]

边界	实际职责	不要误解为
MissionController	生成/载入计划、派发、合并、预算、终局验证、停止	另一个常驻 LLM 或第二套 Bus 控制面
ClosedLoop	子任务状态推进、告警、L0、审计裁决、Gate、历史恢复	所有路径都必须经过 Planner LLM
StateStore	Task/Mission/事件/动作/验证记录、计数与锁	仅复制 state.db 即完整可迁移
AOAdapter	公开 REST/SSE 读取；还承担 approval resolution POST	严格纯只读客户端
ActionExecutor	有限 spawn/send/kill、预算与调用结果记录	任意 shell executor
Planner / Auditor / Verifier	共享 headless Codex 结构化调用；角色输入/输出不同	三个 AO Session 或互相自由聊天的 Agent
Observer / Gate	确定性观察和命令验收	依赖模型判定的监控
事件投影 / Panel	StateStore 后置视图、时间线、memory/project 摘要	预算、恢复或 verdict 的权威来源

必须保留的细节

“单一控制面”不等于“每条 Worker 消息都由 Planner LLM 撰写”：L0 是 ClosedLoop 的确定性提示；用户也可经 directive 通道直接发给 Worker。它们仍通过 Controller/Executor 的程序路径执行。[S05][S06]

“单一正式 Gate 模块”也不等于所有测试执行都通过该模块：Mission baseline failure capture 仍有独立 subprocess 路径。这个例外需要在可靠性工作中统一完整性和证据语义，而不是把旧 Gate 模块搬回来。[S05][S11]

03 三条工作流与成功的含义

默认干净路径：不为简单任务强行调用全部角色

用户任务 → shared preflight → 确定性单任务计划 → AO Codex Worker → Task Gate → Task DONE → materialization commit → integration merge → Final Gate → Mission Verifier → MISSION_DONE

该路径不调用 decomposition Planner、Completion Auditor、completion Planner 或 Task Verifier。Worker 自检、pristine baseline capture 可能另外运行命令，因此“两个正式 Gate”不等于整个 Mission 仅运行两次测试命令。[S05][S06][S14]

异常路径：保留语义诊断，但不靠增加 Agent 解决问题

REPEATED_ERROR / NO_PROGRESS、Gate 失败或证据不足，进入 Auditor → Planner → CONTINUE / SEND_LOCAL_FIX / REPLAN_SPAWN / CANDIDATE_DONE / HUMAN。active turn 的 REPEATED_ERROR 可延期；真正新 error 可触发一次受限 L0。预算耗尽、无法验收、合并冲突进入人工处理。[S06][S07][S17]

恢复路径：已实现一部分，不能泛称“任意续跑”

进程在非终态中断后，可从已持久化 plan/tasks/merged 和 pending 状态继续；历史 VERIFIER_PENDING 有兼容恢复路径。用户 Stop 则把 Mission 置为吸收终态 HUMAN，不能通过重新载入同一行自动重新开工。GUI 的“可续跑”文案需要区分崩溃恢复与用户取消。[S05—S07][S20][S21]

状态或概念	准确含义
Worker 回复 DONE	只是 Worker 自报；不能直接判 Mission 成功。
Task DONE	该子任务当前 Gate 通过；还没代表已经完成合并和终局复核。
MISSION_DONE	Controller 终局条件通过；输出在 integration，不代表用户主分支已更新。
artifact-aware clean	忽略定义内构建产物后无非产物变更；不等于原始 git status 必为空。
Verifier PASS	模型的结构化 verdict；仍必须由程序校验一致性、AC 覆盖和证据完整性。

04 v0.2 功能清单：已实现与已验证

能力	发布版实现	验证与限制
Panel / CLI	共享 MissionRuntime 与 preflight	最终两个入口均有 live PASS。
默认单 Worker	max_subtasks=1 确定性生成 S1	实际最终样例覆盖；不是 LLM decomposition。
显式最多两任务	max_subtasks=2 调 Planner，可返回 1—2	有实现与既有测试；最终发布彩排只覆盖 1。
项目选择	AO registry → selector，显示 path/kind	GUI 已确认；仍会列出无法通过 Git preflight 的项目。
启动预检	Python/Git/identity/AO/Codex/model config/remote ref	缺失前提在 runtime 之前报错；不验证模型在线质量。
自动观察	REPEATED_ERROR、NO_PROGRESS、去重/延期	已覆盖历史重放故障；冷启动统计窗口仍有恢复限制。
异常处理	审计、规划、局部修复、重规划、预算	存在路径；最终 happy-path 没有再触发这些角色。
确定性 Gate	argv、timeout、Git 内容快照与 fail-closed	Task 初始 dirty 合法；Final 初始须 artifact-aware clean。
Git materialization	提交 Worker 改动、integration 本地 merge	最终 live 验证输出正确，未自动更新 target main。
Mission 终局 Verifier	共享 Codex JSON runner、终局复核	真实 PASS；结果一致性仍存在本轮发现的缺口。

对外介绍应当说“默认单 Worker 的闭环任务控制器，支持按需双任务”，而不是将多角色数量或同时常驻作为价值。真正价值是有边界地执行、保存证据、控制失败和交付可检查结果。

证据：[S01][S03—S17]。表中的“有实现”不表示每个分支都在全新电脑上跑过最终 live。

04 交互、诊断与交付功能的边界

能力	v0.2 有什么	还没有或需补强
GUI 实时更新	EventSource /api/stream，约每 2 秒推送 snapshot	不是没有 SSE；需增量/排序/断连状态和更可靠错误反馈。
历史查看与恢复	任务列表、attach、非终态 resume	HUMAN 不是暂停；attach 仍组装完整 runtime，非纯文件零写。
用户指令	按角色/Worker 投递并镜像 Planner	缺持久 ACK、实际生效提示；默认 final Verifier 没消费 role notes。
诊断	SPAWN_FAILURE 脱敏、LOOP_ERROR、预算原因	GUI 对 summary/error 字段覆盖不足，Gate API 查询有缺陷。
结果交付	runtime/integration + 状态和日志	缺“打开结果/查看 diff/导出 patch/显式应用”完整体验。
模型选择	YAML 中各语义角色与 worker.model 可配置	没有 provider 下拉、凭据管理、第二供应商实测，不能只改名称切 GLM/Kimi。
安装/分发	白名单 ZIP、bootstrap、checksum	不是 MSI/EXE；外部工具安装和 AO 注册仍由用户负责。
隐私/安全	包不含开发 profile；有限日志脱敏；loopback	不是完整日志脱敏、危险审批或本地网页攻击防护的证明。

用户体验与 GUI 是否放到 v0.3 ?

是，但要区分“美化”和“正确展示”。首次使用向导、任务模板、模型设置、结果中心、响应式布局适合 v0.3；错误被吞、Gate 证据缺失、权限错误或误判成功，属于当前正确性缺陷，建议先修小版本。

不应丢掉 v0.2 的产品价值

之前的净化发布、non-Git cwd 修复、remote-backed preflight、单任务快速路径都解决了真实问题。v0.3 应在这些能力上补用户旅程，而不是重新堆 Coordinator、消息中间件或第二个状态库。[S01][S04][S23]

05 原始报告闭环：不能写“全部清零”

原始问题	当前结论	v0.3 处理
AO data/profile、.venv、runtime 混入包	本轮产品包边界已解决	继续 release hygiene 与 hash 门禁。
捆绑 AO 再分发问题	改为外部依赖，避免本包捆绑 AO	仍需保留来源/许可说明，不自动授予新许可证。
Bus 与实际控制路径不一致	主干已按 Controller + Store + projection 收敛	GUI 图示进一步显示真实角色调用与非调用。
重复 Client/Observer/Gate/CLI /protocol	当前重复源已退休，历史副本不进包	不为删闲置符号重开大规模清理。
默认 ≥2 Worker / Verifier 过重	默认 1、gate-first、正常 final-only 已完成	高风险复核是后续显式策略，不是当前实现。
Demo 项目硬绑定	Project fallback 移除并有 selector	表单路径/Gate 默认仍是 Python demo 风格。
自动主分支合并/push	隐式行为移除并有 live 证据	只增加显式、有确认的结果应用/PR 操作。
危险自动审批	不能算解决；源码仍有宽松规则	放在 v0.2.1 / v0.3 首批，不等视觉改版。
fingerprint / thread revision / ESCALATE	保留或延期；部分属于投影层	区分真正控制事件与投影，不让旧总线重新控制。
未接线配置、精确时间承诺	部分仍不一致	统一 effective config 与消费者测试。

原始报告把隐私交付列为 P0，把协议/配置列为后续项。本轮核心交付已显著改善；但新源码审计要重新看安全与正确性，而不能因为阶段状态写成 R5 COMPLETE 就把所有 K 项改成 RESOLVED。[S02][S03]

历史验收结论保持原样；本报告新增的问题单独编号 A01—A12。修订判断时保留“为何过去通过、哪个新场景现在失败”的证据，避免把历史成功或失败改写掉。

06 高优先级 A01：审批与路径授权

发现：生产命令审批由 `ClosedLoop._is_gate_command / _is_safe_segment` 判断，并非只由 `approvals.py` 的同名 “safe” 辅助函数判断。当前允许 `git restore/checkout`；Python 模块使用 `startswith("-m pytest")`；`gate` 命令双向前缀匹配；先折叠所有空白再拦截危险分隔符。[S06][S15]

隔离输入（只判断，不执行）	判定	风险
<code>git restore app.py</code>	True	允许可能丢弃修改的操作。
<code>git checkout -- app.py</code>	True	命令注释所称“只读/记账”边界不成立。
<code>python -m pytest_unexpected</code>	True	并非 exact <code>pytest</code> 模块，也被接受。
<code>git status + 换行 + 未知命令</code>	True	判定前空白折叠，原审批请求可能仍保留换行。
<code>python</code> （ <code>gate</code> 为 <code>python -m pytest -q</code> ）	True	反向前缀允许过于短的命令。

文件授权也有缺口：`_rel_to_root` 解析失败时返回原路径，而 `path_matches` 继续做 `glob`。当 `allowed_paths=["***"]`，工作树外的绝对文件也可匹配。隔离探针复现了 True；未实际编辑外部文件。[S15][S28]

建：窄丁，而不是再造沙箱架

路径必须先成功解析并确认包含于允许的根目录；失败返回 `DENY/UNKNOWN`，不再把原始路径交给 `glob`。命令按实际平台与工具输入解析为结构化 `argv`；默认不批准 `restore/checkout`、任意模块、重定向或命令链。明确区分用户预先授权的 `Gate` 与 `Worker` 提议的命令。

验收：工作树外文件、符号链接越界、换行、前缀伪装、危险 Git 操作均拒绝；允许的文件 `Edit` 和精确 `Gate` 仍可一次运行。当前用户没有触发这些路径，不代表它们安全。

优先级：P1 高。建议 v0.2.1；在修复前仅用于可信、可丢弃目标仓库，保留人工监督。单个 `isolated predicate` 复演不是完整 AO exploit 演示。

06 A02：终局 PASS 还缺程序级一致性

发现：MissionController._final_verify 的成功条件是 res.verdict == "PASS" 且 gate_clean。它没有复用历史 Task Verifier 对“顶层 PASS、分项 AC FAIL”的一致性检查，也没有确保 AC 集完整、anti-gaming 无 FAIL、ID 与本次请求一致。[S05][S06][S09][S10]

隔离复演：gate_clean=True；顶层 verdict="PASS"；AC-01="FAIL"；anti_gaming="FAIL"。当前 Mission 终局判定条件仍为 True。这里只复演了源码裁决条件，并未声称真实模型本次输出了这个矛盾对象。[S28]

为什么 strict schema 不能代替这个检查

结果 schema 允许顶层 PASS 和分项 FAIL 同时存在；ac_checks 没有“必须覆盖当前全部 AC”的约束。Provider 的 setdefault 只补缺失 ID，不拒绝错误的现有 ID；_coerce 可把缺失列表变成空列表。此外发布 requirements 没有 jsonschema，fallback 校验弱于完整 schema。[S07][S09][S10][S24]

OpenAI 的结构化输出机制用于格式约束，不能证明业务结论正确。本产品还需要请求关联、证据充分性和 verdict 语义规则，尤其在将来接入只提供 JSON mode 的供应商时。[S26][S27]

最小修复目标

检查	不满足时
verify_id 与本次请求一致；Mission 与 Task ID 映射明确	协议错误，记录有界诊断，不接受其它任务结果。
每个必需 AC 恰好有一个有效 verdict	缺失、重复或未知 AC 不得当 PASS。
顶层 PASS 与全部必需 AC PASS、anti-gaming 无失败一致	矛盾结果不得进入 MISSION_DONE。
确定性路径/完整性失败不能由模型 PASS 覆盖	Controller 直接拒绝；不再依赖模型看出。
证据截断或缺失可见	要求补证据或 HUMAN，而不是静默当完整复核。

建议在现有 contract/Verifier 边界实现一个共享校验器，不新增第二套 Verifier。不因一次模型结果矛盾而改变测试、放宽 Gate 或无限重试。优先级 P1，建议进入 v0.2.1。

06 A03：Git 路径范围存在漏检

重命名只记录新路径，旧路径丢失

worktree.changed_paths 处理 R100 old new 时，分支中再次添加 parts[-1]，即新路径，而不是把旧路径也加入。真实临时 Git 仓库从 protected.txt 重命名到 allowed.txt 后，Git 报 R100，但函数提取逻辑只保留 allowed.txt。禁止路径原文件被移走可能不进入 path gate。[S12][S28]

构建产物规则采用子串匹配

_is_artifact 使用“任一 marker 出现在路径中”判断。隔离探针中 src/data.pyconfig 与 src/.coverage_policy.py 都被当作 artifact，尽管它们可以是真正源码/配置；该规则被路径检查和 Gate snapshot 复用，复用不等于规则本身准确。[S12][S28]

证据采集本身还会改 index

git_diff_text 为展示 untracked 文件执行 git add -N，再 git reset -q 恢复。这与“只读采证”的理想边界不一致，且执行中断、并发编辑或恢复失败会留下状态不确定性。它与新增的只读 Git snapshot 是两条不同路径。[S12]

修复方向	必须保留的正向行为
用 -z 路径输出；rename 记录 old/new，或明确 --no-renames	支持空格、中文、特殊路径，删除/新增不漏报。
artifact 按目录段或明确后缀判断	__pycache__、.pytest_cache、*.pyc 等正常产物仍不误报。
untracked diff 不修改真实 index	读取新文件内容并标记证据来源，异常显式失败。
Final Gate 后确定性校验 Mission 范围	模型不承担最后一道路径范围裁决。

优先级 P1。建议先修 rename / artifact 判定，再处理只读采证与最终范围复核。测试“新增、删除、重命名、同路径改内容、stage/unstage”应覆盖完整组合，而非只验证普通 app.py 修改。

07 A04 / A05：GUI 证据与本地 API

A04：Gate evidence 的空列表不是“没有运行 Gate”

Panel.snapshot 对 gate_runs 调用通用 _payloads，SQL 查询 payload_json；但 StateStore.gate_runs 保存的是 command、cwd、exit_code、stdout 等标量列，没有 payload_json。_rows 将 OperationalError 重试后吞成 []。隔离 SQLite 探针有 1 条 Gate 记录，仍得到 no such column: payload_json。[S13][S20][S28]

因此此前 GUI 中能看到 GATE_EVIDENCE 投影或 transition，不等于专用 gate_runs API 正确。新增 SPAWN_FAILURE 写 summary、LOOP_ERROR 写 error，而前端 alert 主要显示 description；mission.reason / panel_errors 也没有完整可视化。需要补 schema-aware 查询和端到端渲染测试。[S14][S19—S21]

A05：loopback 限制不等于完整本地控制防护

Server 仅绑定 127.0.0.1，这是正确默认值；但写 API 未检查 Host/Origin、会话 nonce 或 CSRF，_body 也未严格要求 JSON Content-Type。_attach/_resume 拼接外部提供的 mission_id，缺少目录包含性检查。前端 esc 没有转义引号，却被用于 title/data-* 属性拼接。[S20][S21]

隔离字符串复演能看到引号逃出 title 属性。本轮未执行浏览器攻击，不能宣称已经证明跨站远程执行；但这些是应在扩大分发前补齐的明确输入与页面边界。单纯添加“本地安全”文案不构成修复。[S28]

建议优先级	修改与验收
P1 / 小补丁	Gate 专用查询；真实失败原因可见；错误不转成“无记录”。
P1 / 先本地复现	Host/Origin/nonce、Content-Type、mission_id 规范与 root containment；跨源请求拒绝。
P1 / GUI 安全	用 textContent / DOM API 或上下文安全转义，危险文本只作为文本展示。

08 A06—A08：恢复、指令与交付一致性

发现	当前事实与影响	建议
A06 用户指令“已发送”≠已生效	queue 先入内存后写 JSONL；磁盘失败时可能已排队。默认 final-only Mission Verifier 未消费 per-task verifier notes。初始 Worker prompt 没有完整路径清单或 user_instruction。	持久 command ID、received/applied/rejected 状态；路由到真正消费者；无消费者明确禁用。
A07 外部动作非严格 exactly-once	AO spawn/send 成功与 SQLite mark 之间有崩溃窗口；超时/解析 Session ID 失败也可能留下未知外部结果。	operation intent + 对账；先判断 AO 是否已创建，不因未知直接重发。
A07 停止确认不足	kill_worker 返回 bool，合并/终态清理多处 best-effort 忽略失败；仍可能提交尚在活动的工作树。	停止成功/已终止的外部事实作为 materialization 前提；失败保留现场。
A08 Stop 与 Resume 含义不一致	Stop 将 Mission 写 HUMAN 吸收终态；UI toast 却称“状态可续跑”。	区分取消、挂起、崩溃恢复；人工重试创建关联新 attempt，旧终态不可改写。

恢复材料不是只有 state.db

逻辑状态的权威仍是 StateStore，但 frozen base 与 baseline failure 存在 JSON sidecar；integration 是依赖目标仓库 common Git directory 的 linked worktree。直接移动 runtime 或删除目标仓库不保证保留可用交付。v0.3 应提供可验证导出与完整 recovery manifest，而非让用户猜文件夹。[S05][S12][S13]

终局重试需要 evidence revision

当前 Mission Final Verifier 异常会跨 tick 重跑 Final Gate，并生成新 verify_id。建议以 integration HEAD + Gate evidence digest + config revision 标识一次终局验收，复用已完成且仍有效的确定性证据；仅重试失败部分。不得复用已经变化的代码证据。[S05]

08 A09：多 Worker 与基线语义

当前 `max_subtasks=2` 的实现可安排两个子任务，AO Worker 可在服务端并行；但 Controller 本地按任务逐个 step。某一 Task 的同步 LLM/Gate 调用会阻塞另一个 Task 的下次观察，不是异步多任务调度器。[S05][S06]

依赖完成，不代表依赖代码已进入新 Worker

`_dispatch_ready` 以依赖 Task DONE 为条件，且调用顺序在 `_merge_done` 之前；新 Worker 仍由 AO Project 默认分支启动，没有把已经完成依赖的 integration commit 传给它。因此有依赖的 S2 可能看不到 S1 成果。此为静态调用顺序风险，尚未在本轮 AO live 复现。[S05][S14]

统一“验收基线”的语义

Worker 通常来自 remote-backed AO base，而 integration 起点取目标主 worktree 当时 HEAD。若本地主分支与 origin 分叉，或用户在运行中推进主分支，二者未必相同。只核对最终 `changed_paths` 与 Gate，不足以解释差异属于任务还是外部变化。[S04][S05][S12]

v0.3 建议 contract	验证场景
Mission start 固定 source commit、配置 revision 和任务范围	target main/origin 不一致时明确选择或拒绝，不能静默混用。
独立双任务允许共同 base 并行	路径不相交、合并正确；两个 Session 独立观察与预算。
有依赖任务从已验证 dependency commit 开始	S2 直接调用 S1 新增 API，不允许重新“自己实现一遍”通过样例。
冲突/外部漂移保留证据并人工决定	不得自动 checkout/reset 用户修改，不将冲突当模型失败。

v0.2 对外仍应优先默认单 Worker；不能把“最终单任务 live 通过”推广成所有 dependency graph 已验证。v0.3 不要为增加并发数量牺牲基线与合并的可解释性。

09 A10：有效配置、性能与调用成本

配置/路径	源码事实	改进方向
roles.worker.model vs worker.model	正式 Executor 读取 worker.model；重复角色键易误导	一个有效键；UI 显示消费者与最终值。
gate.timeout/output_limit	YAML 有键；Gate 仍固定 300 秒、单流截断 8000 字符	删除无效键或真正接线；边界测试。
Panel 时间参数	Panel 用自己默认值覆盖 YAML，并做 int/min1 钳制	输入校验与文案一致，保存 effective config。
AO 每 tick “一次采集”	一次 Python 方法调用内部是 session list + 各 Session conversation 请求	按请求数与字节测量，不把方法调用当一次 HTTP。
SSE snapshot / history	已有 SSE；每连接每两秒查状态，JSONL 读取全文件后取 tail	增量 cursor、限量查询、共享快照与可恢复连接。
LLM transport	角色调用同步；推理、重连、retry 混入等待时间	分阶段计时、实际 profile/model、重试原因分开记录。

用户看到“简单函数用了数分钟”并不能推出都是 Codex 慢。最终 CLI 的 418.4 秒包括 AO 启动、Worker、模型重连、轮询、Git/Gate 和 Mission Verifier；只有拿到每次调用的起止、等待、重试与输入长度，才能判断瓶颈。[S01][S04—S06][S16]

历史样例 776.9s → 646.1s → 409.5s 是不同阶段、不同网络下的单次观察，不是严谨 benchmark。结构性确定收益是“减少不必要角色调用”；墙钟差额不能全部归因于架构或模型。[S03]

不要把无效等待变成更多 Agent

先增加可归因的 metrics 和 UI 进度，再考虑并发/缓存。共享 runner 能承载 transport tracing；StateStore 承载事实，前端只展示。Provider 错误应区分认证、配额、网络、协议、语义拒绝，不把所有错误都继续重试。

10 GUI 审计：应从拓扑演示转向任务完成

v0.2 的面板已具有真实 Project selector、表单、历史任务、状态拓扑、指令栏、时间参数、证据与 SSE。问题不是“没有 GUI”，而是日常用户最关心的目标、等待原因、下一步与结果被大拓扑和机器状态词挤到次要位置。[S20][S21]

用户问题	当前障碍	v0.3 界面要求
我能运行了吗？	AO/Codex 外部前提和 Git ref 规则复杂	首页就绪清单，失败项附明确修复入口；不要求读日志。
到底操作哪个项目？	首项自动选中；Scratch 等无效类型也显示	要求显式确认项目；显示 source commit、范围与风险。
我该填什么？	默认 app.py / math2.py / pytest，不适用多数仓库	模板只是建议，用户确认后生效；空范围不静默放宽。
为何一直等？	只显示累计时间、最后 actor；错误 toast 易消失	当前阶段、正在调用谁、重试次数、阻塞原因、预计非承诺式说明。
Gate 过了为何没完成？	Task DONE 与 Mission DONE 混淆	分离 Task、集成、Final Gate、Final Verifier 四级验收。
成果在哪，怎么拿走？	integration 路径/主分支未变让用户误以为没保存	结果中心：diff、commit、测试、打开目录、导出 patch、显式应用。
我能停止后再继续吗？	取消与恢复的文案不一致	按钮按真实状态提供取消/恢复/创建重试，不发送假成功。

基础布局是固定三列（300px / 1fr / 380px），小屏和缩放容易拥挤；Gate 输入仍是单行控件但文案称每行一条。时间取原字符串片段，缺本地时区说明。v0.3 应补键盘操作、可读对比度、响应式布局和长证据展开。[S21]

先修 A04 的真实数据与错误展示，再改视觉层级。引入 React/Vue 不是验收目标；只有组件/测试复杂度证明需要时才换框架，不要为“现代化”重写全部 UI。

11 多模型现状：能改 model，不等于能换供应商

维度	v0.2 状态	是否可宣称已支持
Codex 模型 ID	语义角色、Worker 有 model 配置字符串	可配置；只有已验收 ID/环境被证明可用。
Planner/Auditor/Verifier 后端	固定 CodexCli*Provider + run_codex_json	没有 GLM/Kimi 原生 API Provider。
Worker 后端	通过 AO Codex harness，使用 --model	Worker 与语义角色接入边界不同。
登录与凭据	preflight 明确要求 Codex ChatGPT 登录	不是已有多供应商 key/endpoint 管理。
推理档位/profile	没有完整产品配置与展示契约	不能假设 YAML model 覆盖了用户全部 Codex 配置。
模型设置 GUI	没有 provider/model 配置页	应纳入 v0.3。

模型名、供应商、API 协议、认证方式、工具执行 harness 是五件不同的事。直接把 gpt-5.6-sol 改成某个 GLM/Kimi ID，不能证明请求会发往正确服务、结构化输出会按同一规则返回，更不能证明 AO Worker 有相同的工具/审批能力。[S04][S08][S14][S22]

官方资料可支持的范围

OpenAI 配置文档提供自定义 provider/profile 等能力，但具体版本的协议、认证与 override 规则仍要验收。Z.AI 文档提供 json_object 模式及应用侧 schema 校验示例；这不等于它和 Codex CLI 的结构化传输/工具调用完全互换。本轮没有取得足以确认 Kimi 指定模型全部能力的官方实测依据，故仅列候选。[S25—S27]

“国产模型更快”是待测假设，不是规划前提。地域、服务配额、推理档位、上下文长度、重连与协议适配都可能改变总时间；必须用同一任务与验收规则比较。

12 v0.3 模型接入方案：角色不增殖

保留现有 PlannerProvider / AuditorProvider / VerifierProvider 语义接口，把供应商差异放到薄 transport adapter。第一期只接一个经过验证的新供应商；不要同时承诺所有 GLM、Kimi、Claude、本地模型和任意代理。[规划]

Role input → 固定证据 envelope → Transport adapter → 供应商响应 → 规范化结果 → 本地 schema / ID / verdict 校验 → Controller

Worker: Controller → ActionExecutor → 经能力验收的 AO harness（独立适配路径）

接入层次	工作范围	发布条件
Level 1: 同 Codex 可用模型/profile	不同角色可选已验证 ID/推理档位；新 Mission 固定配置	不改用户全局 Codex 配置；显示 effective model/profile。
Level 2: 一个语义 API 后端	先替换只读 Auditor/Verifier 或 Planner JSON 调用	schema、拒绝/截断、超时、ID、一致性、脱敏和费用归属验证。
Level 3: 第二 Worker harness	验证 AO 实际支持的工具、编辑、终止、权限与 session model	单独完整 Worker E2E；不能把聊天 API 等同于 coding agent。

配置和数据边界

建议 Mission 保存 provider、model、profile/effort、timeout、retry policy、schema version 与配置摘要。API key 只保存 secret reference，不进入 SQLite、报告、压缩包或 UI 日志；不覆盖 ~/.codex 全局设置。配置更新只对新 Mission 生效，运行中切换必须是显式动作。

禁止默认静默跨供应商 fallback：它可能把用户代码发送到未授权服务，也会改变验收语义与成本。需要切换时，先展示目标服务、数据范围和原因，由用户确认；每次尝试记录真实后端。

不要把所有厂商 raw JSON 传播到 Controller。既有角色输入/输出继续稳定，缺少能力的适配器直接给 capability error，不让模型“猜格式”通过。安全修复和本地验证先于多模型接入。

13 v0.3 信息架构与完整用户旅程

区域	主要内容	验收重点
就绪 / 首页	Python、Git、AO、Codex/凭据状态；新建任务、最近结果	无需发模型请求即可定位缺失前提；不把 AO 未开当产品崩溃。
新建任务	明确 Project、目标、范围、Gate、预算、模型配置摘要	显示会修改哪里；默认 1；高级项渐进展开。
运行工作台	阶段进度、实际角色调用、阻塞原因、取消、审批卡片	低延迟可解释；未调用角色不高亮伪装运行。
结果中心	Mission verdict、AC 清单、Task/Final Gate、diff/commit、导出	回答“做了什么、验证了什么、成果在哪里”。
历史 / 恢复	过滤搜索、终态查看、可恢复检查点、新 attempt 关联	旧记录不被重试覆写；缺外部 worktree 给可读诊断。
设置	有效配置、供应商能力、secret reference、环境诊断	配置值与消费者一致；未支持选项不可被保存为已生效。

建议的新任务确认摘要

“将对项目 X 的提交 Y 创建隔离 Worker；允许修改 A；Gate 为 B；默认只做本地 integration；成功不会自动修改 main；预算/模型为 C。”用户确认的是具体行为，而不是只点击一个不透明的“开始”。

结果应用：显式，不恢复危险自动 push

第一步提供“打开 integration / 查看 diff / 导出 patch”；第二步再提供经确认的应用或创建 PR。动作前重新检查目标 HEAD/dirty、展示冲突与变更，成功后记录目标 commit。默认不自动写回，不能把“发布产品”混同于“替用户发布代码”。

界面测试方法

把 GUI 数据 contract 与浏览器自动化放进开发/CI 环境，不随 runtime 装 Selenium/Playwright。覆盖窄屏、长中文、引号文本、断连、错误码、空状态、失败原因、键盘操作与多阶段终局；至少两名没有参与开发的人完成首次任务。

14 版本安排：先补边界，再做产品化

版本/里程碑	范围	退出标准
v0.2 保持冻结	保留已发布 tag、ZIP、历史验收；明确本地可信项目边界	不改旧 tag，不覆盖已发布附件。
v0.2.1 窄补丁	A01 审批/路径、A02 终局一致性、A03 rename/artifact；A04 Gate 查询；本地请求/转义边界按复现纳入	每项先有失败用例；全量回归与小型独立 CLI/GUI smoke。
v0.3-A 可靠性与统一配置	A06—A10；配置快照、真实错误/指标、恢复/取消与外部动作对账	异常/重启场景通过；schema 和 evidence 契约稳定。
v0.3-B 用户旅程与 GUI	首次使用向导、项目/任务模板、运行与审批、结果中心	新用户自助完成任务并理解结果；浏览器 E2E 与接口一致。
v0.3-C 第二供应商与评测	Level1 配置 + 一个语义 API adapter；Worker 视能力单独准入	质量门禁通过，性能/成本有记录；不静默跨供应商。

A 与 B 可以在契约固定后并行推进。C 必须在本地结果校验和配置生效机制之后，否则多模型只会放大已有模糊行为。GUI “美化” 不应压过 A01/A02；但也不必等所有低优先级内部符号清理完才改善日常使用。

版本范围明确不做

v0.3 不同时建设云多租户、公开远程控制、无限 Worker、向量长期记忆、Agent 插件市场、通用模型网关或完整 Electron 重写。没有直接用户收益和验收场景，不新增 Coordinator、消息队列或第二状态库。

排期按可独立验收的纵向功能切片，而非继续几十次“只读审计→删一个文件”。先将本报告的问题转成可重复测试，再按 patch / reliability / UX / provider 四个工作流管理。

IMPLEMENTABLE BACKLOG

15 优先级 Backlog 与责任边界

编号 / 优先级	目标与现有落点	完成定义
A01 / P1	审批 parser 与 containment; closed_loop / approvals	危险命令、越界、UNKNOWN 均不自动批准；正常文件编辑仍通过。
A02 / P1	终局共享校验; mission_contracts / verifier / mission	错误 ID、缺失 AC、PASS/FAIL 矛盾、反作弊失败无假阳性终局。
A03 / P1	Git 取证; worktree / mission	rename old/new、artifact 精确、只读 diff、final scope gate。
A04 / P1	证据 API; state_store / panel	每条 Gate 可查询；错误显示真实原因与 task/mission scope。
A05 / P1	本地请求/显示边界; panel	Host/Origin/token 与 path 输入测试；特殊文本不进入 HTML 结构。
A06 / P1	指令生效; directives / mission / panel	received→applied/rejected 持久回执；final verifier 能消费其指令。
A07 / P1	外部副作用对账; executor / store / mission	spawn 成功但 ack 中断后不盲目重复；kill 未确认不提交。
A08 / P2	取消/恢复; mission / panel	Stop、暂停、崩溃恢复区分；同一终态不被反复打开。
A09 / P2	双任务基线; mission / executor / worktree	依赖代码进入新 Worker；外部 HEAD 漂移有显式策略。
A10 / P2	配置/性能; run_mission / config / adapter	无无效可调项；模型、轮询、Gate 和重试时间可归因。
A11 / P2	模型能力与证据完整性; existing Providers	统一结果校验、截断标记、凭据隔离与第二后端准入。
A12 / P2	交付体验与维护; panel / builder / docs	结果导出、备份恢复、支持矩阵、clean Windows 验证与 CI。

P1：影响权限、结果可信或关键诊断，优先修复；P2：可靠性/体验的重要改进。优先级基于本轮审计，不表示已观察到真实用户损失或已完成漏洞利用。

DEFINITION OF DONE

16 v0.3 验收矩阵：不能只跑 clamp

测试组	最低验收场景	不能用什么替代
结果正确性	真实通过；Gate 失败；矛盾 PASS；AC 缺失/重复；错误 ID；证据截断	只看模型最后回复或顶层 verdict。
授权与路径	越界文件、符号链接、rename 原路径、产物同名源码、命令链/伪装模块	只对正常 app.py edit 做一次测试。
控制 API / GUI	跨源/非法 Host、mission_id 穿越、引号文本、失败/断连显示	只用 curl 证明 HTTP200。
中断/幂等	spawn 已创建但客户端超时；send 后 ack 前中断；kill 失败；final 记录后进程重启	仅重启空闲 Panel。
停止与恢复	取消生成中/审计中/Gate 中；崩溃恢复；终态只读；新 attempt 关联	把 HUMAN 改回 RUNNING 或覆盖旧记录。
双任务	独立并行、依赖代码、冲突、主分支漂移、单任务长期模型等待	只跑两个互不相关函数就宣称依赖图完成。
模型适配	合法 JSON、schema 错误、拒绝/截断、认证/配额、网络中断、模型不支持能力	仅测“回复 OK”或 HTTP 成功。
首次用户	两台/两套新 Windows 环境；按 README 安装、选择真实项目、看到结果	开发者原.venv 与预配置 profile。
release	从 ZIP 独立启动；checksum；无 profile/runtime；记录实际版本	把仓库源代码 zip 当正式产品包。

438 项既有回归应保留为基线，而不是 KPI。增加测试必须绑定问题和用户旅程；修复后按真实新数量报告。单元/合同测试应离线；真实模型 smoke 控制预算、保留失败分类；GUI 则同时验收页面与 StateStore/AO/Git 事实。

17 性能、质量与可观测性指标

先建立可比基线，再承诺加速

建议建立 20 个代表性任务，每个候选配置至少重复 3 次；包含小修改、新文件、错误修复、长 diff、依赖、网络恢复与验收拒绝。相同 base、Gate、任务范围和成功标准，固定供应商/模型/effort 版本。结果按成功任务与失败任务分开统计，不能用快速失败拉低延迟。

指标	记录方式 / 建议验收
任务成功与假 PASS	Gate/AC/人工复核共同定义；已知负例集假 PASS 必须为 0。
结构化输出	解析率、schema 率、ID 率、coherence 率分开；不能只记录 JSON 率。
任务延迟	p50/p95：启动、Worker、等待、重试、每个 Gate、Verifier 与总时长。
成本	实际调用数、输入/输出 token、供应商费用口径、失败重试成本；未知标 unknown。
UI 响应	建议本地控制 ACK p95<1 秒，事件可见延迟 p95<2 秒；需在目标机实测，不保证外部终止同步完成。
用户操作成本	首次完成任务所需人工步骤、错误修复次数、找到结果耗时；记录依赖下载安装时间。
恢复健康	重复 Worker、遗留活动 turn、重复 side effect、漏记录/被吞错误数量。

所谓“更快模型”只有在质量不退化、费用/数据边界清楚时才值得成为默认。对相同模型降低推理档位也要通过相同验收；不要在没有观测的情况下把 timeout、线程数或重试次数调大当优化。[S25—S27]

日志也有权限边界

记录 phase、operation ID、model/provider、durations、error category、evidence digest；默认不落完整 prompt、凭据或用户 home 路径。现有 spawn 脱敏只是局部修复，不能推导为所有 subprocess/model/event 日志已脱敏。[S08][S14][S20]

18 下一步执行建议与冻结纪律

第一项工作：建立 v0.2.1 可重复失败用例

不直接开始“重做前端”或“接全部国产模型”。先在新分支复验 A01—A04 的最小产品级测试，确认 Windows/AO 字段形态；修复范围限制在现有模块。已发布 v0.2 tag、ZIP、checksum 和历史成功记录保持不变。

建议给 Codex 的第一条任务：

Goal：补 v0.2.1 的权限、终局一致性和证据显示边界，不扩展功能。

Context：冻结 v0.2 + 本报告 A01—A05；保留原始验收与失败证据。

Constraints：不换模型/框架、不改公开 v0.2、不新增控制面、不在用户重要仓库跑破坏性用例。

Done when：先有可重复负例，再有修复；正向单 Worker 回归仍通过；给出 diff、测试分类、剩余风险和小 PR，未审计不发布。

工作方式：减少审批循环，保持证据清晰

每个 issue 给出受影响函数、条件、最小测试、预期行为、支持范围、负责人；只在存在真实不确定性时暂停。无须为每一句低风险文案再做一轮完整模型 E2E。涉及 runtime/Git/审批/契约变更才重跑相关 live；纯报告文件变化用产品 blob 等价性检查。

发布前后范围

v0.2.1 的 release note 应明确“收紧边界，不撤销原有成功样例”；v0.3 说明新增模型究竟覆盖哪些角色、哪种认证和哪个 AO harness。保留 rollback 版本，但不自动回退数据格式或覆盖用户 runtime。公开发布前还应核对项目与历史移植代码的许可/归属，不因 fork 或新品牌自行添加许可证。

当前首要用户安全建议是：保持 loopback、可信项目和人工监督，备份重要改动；不将本地比赛版接入公网或作为任意外来仓库的无人值守执行器。这是风险控制建议，不是新增代码功能。

最终决策：UX、模型切换和 GUI 确实属于第三版主线；但第三版的定义应是“可靠且可理解地完成任务”，而不是仅增加供应商下拉菜单与视觉样式。

附录 A 本轮隔离复演结果

环境：本次分析容器（Linux），只使用临时目录、SQLite 内存表、源码提取的函数/判定分支。未运行整个 CLAO 测试套件；未发 AO/Codex 请求；未执行被测审批字符串；未执行浏览器攻击。结果足以定位规则缺口，不替代 Windows 产品级回归。[S28]

探针	实际输出	可支持的结论
A01 命令审批	restore / checkout / pytest_unexpected / 换行文本 / 短前缀 python 均 True	当前审批谓词过宽；是否执行依赖真实 AO 请求与 shell，不能虚构已遭利用。
A01 工作树外路径 + **	root 外绝对路径 allow=True	containment 失败后 glob 仍可能放行；需产品级文件审批测试。
A02 终局条件	PASS 顶层 + AC/anti FAIL + gate_clean → 条件 True	final predicate 没有 coherence 防线；非本次真实模型输出。
A03 Git rename	Git:R100 protected.txt allowed.txt；提取: [allowed.txt]	实际临时 Git 输出能复现 rename 原路径遗漏。
A03 artifact	src/data.pyconfig 与 src/.coverage_policy.py → True	子串分类可以隐藏真实源码/配置路径。
A04 Gate 查询	1 row；SELECT payload_json → no such column	后端通用 JSON 查询与真实 gate_runs 表不相容。
A05 HTML 属性转义	输入引号保留，生成额外属性片段	转义不适用属性上下文；未宣称浏览器已执行脚本。

以上探针按公开发布版对应函数逐段复演；在未使用的 cd 分支中采用拒绝 stub，不影响列出的 case。Git rename 仅创建/删除隔离测试目录，未触碰用户源码或工作仓库。

本轮尚需产品场景确认的推断

跨源浏览器到本地 API 的实际可达性；依赖 S2 是否缺 S1 代码；AO timeout 后 Session 创建对账；kill 失败时合并；多窗口并发 Mission ID；进程恢复后 Observer 窗口/指令丢失。它们应形成专用测试，不把“可能”写成“已发生”。

附录 B 文档解释与验收边界

为什么旧结论 PASS 与本轮新问题可以同时成立

过去的 PR 审计大多是范围限定的改动审核：是否只删除无调用旧模块、是否保留只读 sandbox、是否修复某个真实异常。最终 live 采用指定函数和默认单 Worker 流程。它们并未覆盖本轮的矛盾 Verifier 对象、特殊路径、宽通配授权、浏览器属性输入或外部副作用崩溃窗口。

因此发布成功不必撤回；但“所有问题都已修改完成”的口径应继续收窄。本轮报告的价值是把下一版要解决的风险变成可验证任务，而非继续依赖总测试数量或演示成功。

不要混淆的五组概念

概念一	概念二	关系
消息投影线程	控制状态机	旧 envelope/thread 的修订不是自动变成生产裁决的必要条件。
模型 ID 切换	供应商/Worker 切换	名称字符串不能代替协议、工具、认证和能力适配。
read-only Codex sandbox	全过程完全无副作用	Gate 会执行命令，Worker 会写文件，证据采集还可能修改 index。
Git clean snapshot	测试没有任何副作用	忽略产物且只看受检查 repo；不是网络/外部路径的沙箱。
SSE/API 返回成功	用户页面正确且动作生效	仍需匹配表结构、字段、状态语义和真实消费者。

CI 与验证责任

438 项本机/解压包回归是有效已有证据，但本轮没有取得独立 CI 运行、平台覆盖或测试覆盖率报告。v0.3 应在开发工作流固化离线 unit/contract tests、打包 smoke、浏览器 E2E 和受控 live 矩阵；不能把现有计数直接称为跨平台 CI 保证。

报告不要求恢复历史重复实现。可复用的已有模块已经足够：修 contract、查询、路径规则和可靠性协议；只有明确不可替代能力才引入新依赖。

PINNED SOURCES

附录 C 证据与来源索引（1/3）

源码链接固定到已发布提交，不随 main 漂移。文中 [Sxx] 既指数据来源，也限定结论的范围。外部模型文档仅用于规划可行性，不能替代本项目后端与指定版本实测。

[S01] 已发布 v0.2 Release 与附件元数据

<https://github.com/zhaoshiyi4246/agent-orchestrator-AI-worker/releases/tag/v0.2>

发布状态、包名、SHA-256、支持范围和最终验收记录。

[S02] 原始架构与交付审计报告（用户附件，2026-08-31）

文件：闭环多智能体系统_v0.2_架构与交付审计报告(5).docx；第 4—11 节及优先级表。

[S03] 发布基线 PLANS / 当前问题登记

[固定发布源码](#) · [PLANS.md](#)

K1—K18、R2/R5 收尾、最终 CLI/GUI 证据；历史阶段描述不代表全量能力。

[S04] run_mission.py

[固定发布源码](#) · [交付/closed-loop-v2/run_mission.py](#)

mission_preflight、build_runtime、MissionRuntime、环境和模型组装。

[S05] mission.py

[固定发布源码](#) · [交付/closed-loop-v2/src/loopcore/mission.py](#)

确定性计划、dispatch、hydrate、merge、baseline、_final_verify、stop。

[S06] closed_loop.py

[固定发布源码](#) · [交付/closed-loop-v2/src/loopcore/closed_loop.py](#)

告警/L0/gate-first、自动审批、历史 Task Verifier、失败恢复。

[S07] mission_contracts.py

[固定发布源码](#) · [交付/closed-loop-v2/src/loopcore/mission_contracts.py](#)

Task/Mission 契约、合法状态转移、可选 jsonschema 与 fallback。

[S08] codex_cli.py

[固定发布源码](#) · [交付/closed-loop-v2/src/loopcore/codex_cli.py](#)

结构化 Codex CLI 共用边界、stdin、read-only、non-Git cwd。

[S09] verifier.py

[固定发布源码](#) · [交付/closed-loop-v2/src/loopcore/verifier.py](#)

VerifierInput 截断、_coerce、ID setdefault、结果校验。

[S10] verifier-result.schema.json

[固定发布源码](#) · [交付/closed-loop-v2/schemas/verifier-result.schema.json](#)

结果枚举；不保证 AC 集完整性与顶层/分项 verdict 一致。

附录 C 证据与来源索引 (2/3)

[S11] mission_gate.py

固定发布源码 · 交付/closed-loop-v2/src/loopcore/mission_gate.py

argv 执行、GateRun、完整性快照、硬编码 timeout/output limit。

[S12] worktree.py

固定发布源码 · 交付/closed-loop-v2/src/loopcore/worktree.py

changed_paths、artifact 分类、snapshot、diff、commit、integration。

[S13] state_store.py

固定发布源码 · 交付/closed-loop-v2/src/loopcore/state_store.py

SQLite 表结构、WAL、锁、Gate 标量列、记录与幂等标记。

[S14] action_executor.py

固定发布源码 · 交付/closed-loop-v2/src/loopcore/action_executor.py

Worker spawn/send/kill、failure diagnostics、预算、prompt、action ack。

[S15] approvals.py

固定发布源码 · 交付/closed-loop-v2/src/loopcore/approvals.py

文件路径约束；命令自动审批主路径须结合 closed_loop.py 判断。

[S16] ao_adapter.py

固定发布源码 · 交付/closed-loop-v2/src/loopcore/ao_adapter.py

AO REST、approval POST、workspace endpoint、全量事件聚合。

[S17] event_observer.py

固定发布源码 · 交付/closed-loop-v2/src/loopcore/event_observer.py

REPEATED_ERROR / NO_PROGRESS、事件去重与内存窗口。

[S18] envelope.py

固定发布源码 · 交付/closed-loop-v2/src/loopcore/envelope.py

投影消息枚举、路由、含 source 的 issue_fingerprint。

[S19] bus_projector.py

固定发布源码 · 交付/closed-loop-v2/src/loopcore/bus_projector.py

StateStore 后置投影、按表 cursor、Gate/Verifier/Final report 投影。

[S20] panel/server.py

固定发布源码 · 交付/closed-loop-v2/panel/server.py

本地 HTTP/SSE、表单、snapshot、gate_runs 查询、stop/resume/directive。

PINNED SOURCES

附录 C 证据与来源索引（3/3）

[S21] panel/index.html

固定发布源码 · 交付/closed-loop-v2/panel/index.html

GUI 布局、SSE、字段渲染、esc、模型界面缺口、错误反馈。

[S22] config/default.yaml

固定发布源码 · 交付/closed-loop-v2/config/default.yaml

重复模型键、阈值与未接线配置；需结合消费方验证。

[S23] 发布 manifest / builder

固定发布源码 · 交付/release-manifest.txt

SOURCE => DESTINATION 白名单；builder：同目录 build-release.ps1。

[S24] 产品 README / bootstrap / requirements

固定发布源码 · 交付/closed-loop-v2/README.md

用户契约；同目录 bootstrap.ps1、requirements.txt、启动 CLAO.bat。

[S25] OpenAI Codex 配置参考（检索日 2026-09-05）

<https://developers.openai.com/codex/config-reference/>

供应商/模型/profile 是不同维度；当前官方页面不替代 CLI 0.150.1 兼容实测。

[S26] OpenAI Structured Outputs 指南

<https://developers.openai.com/api/docs/guides/structured-outputs>

区分 JSON、schema 与业务语义正确性。

[S27] Z.AI Structured Output 官方文档

<https://docs.z.ai/guides/capabilities/struct-output>

json_object 模式及应用侧 schema 校验示例；不构成速度或 Worker 兼容承诺。

[S28] 本轮隔离探针记录（2026-09-05）

提取发布版函数/SQL/裁决条件，在 Linux 临时目录复演；非完整项目回归，无 AO/模型/浏览器攻击调用。详见附录。

检索与引用说明

审计源代码基线：4d3e8e6b5e70bab868b2eef0d28c7742dea044ba；外部资料检索日：2026-09-05。Kimi 仅为候选供应商，本轮未取得足够官方能力证据，未给出“可直接替换”结论。

原始附件保留其第 4—11 节组织与“如无必要，勿增实体”原则；本报告的 A01—A12 和版本排期为新增审计/规划，不反写原报告。

本报告未访问用户凭据、未修改仓库、未重打包或覆写已发布 v0.2。